Supplementary Material: Proofs of Whole Inventories, Charges, and Source-Native Continuation

Complete Proofs: Whole Inventories, Charges, and Source-Native Continuation

Jian Gao

Independent researcher · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

Abstract

This supplement develops every complete interface used in Sections 4, 5, 7, and 8 of the main text: the four mathematical dispositions of the original receipt; lifting every inventory in Type through splitting and merging; the complete transported remainder; source syntax and paid relations; integer coefficients and binding bodies; prefixes and actual calls; admission at every count; typed inventories; actual content updates; and all cofinal segments of the same runtime. Complete proofs from the original-root residual to payment and a completed answer, and from a long calculation to its next correction, appear in Sections 9 and 14 of the main text, respectively. Shared theorems are cited from the published second version of the process core, DOI 10.5281/zenodo.23297175. Here we develop their actual consumption in the whole ledger.

A Four Dispositions of the Same Original Receipt

A.1 Complete Recovery and Three Exact Residuals

Fix an original dynamical source D, an actual state s, and an additive commutative target group A. At each original event, the source input law supplies a generator type G, a complete exposure rooted at that occurrence, a seed, a continuation tree, and an evaluator tree. The seed and continuation tree generate the complete history H_s. These inputs contain no disposition branch, zero-kernel premise, coverage premise, inverse map, target, or next state.

The decisive source action occurs first: eliminate the result of the same original whole-ledger compilation. Its nativeWrite payload is the original $\epsilon = D.\mathrm{emit}(s)$; the other four structural payload types are PEmpty in this named vocabulary. Read the source seed, continuation, and evaluator through this ϵ. The round-trip equality of the original event presentation gives receiptFromCompiler=ϵ. Thus all the algebraic objects below arise from the same transfer receipt in the old ledger.

Let $E : \mathbb{Z}^{(G)} \to A$ be the linear extension of the original evaluator. The source history retains the closure of actual generators and the relation closure. When the relations are sound, E annihilates them and generates a map $\overline{E} : \widehat{H}_s \to A$ on the completed carrier. Otherwise, there is an actual relation q in the source relation closure with $E(q) \neq 0$. The original faithful settlement first classifies

this soundness; the sound branch then checks its kernel and cokernel internally. The four branches are:

1. **Faithful branch.** The relations are sound, the completed map has zero kernel, and the actual cokernel is a singleton. The source generates a complete linear equivalence $\phi : \widehat{H}_s \simeq A$. The recovery of any $a \in A$ is $\phi^{-1}(a)$, with original evaluation $\overline{E}(\phi^{-1}(a)) = a$. This universal recovery is a positive output.
2. **Unsound-relation branch.** The source generates an original relation q and its exact coordinate $u = E(q) \neq 0$. The relation belongs to this history, and its nonzero image is identified by the original evaluator itself.
3. **Kernel-residual branch.** The relations are sound but $\ker \overline{E} \neq 0$. The source generates $k \in \widehat{H}_s$ with $k \neq 0$ and $\overline{E}(k) = 0$. This is a nonzero coordinate of the source completion; its zero numerical evaluation does not erase it.
4. **Coverage-residual branch.** The relations are sound, the kernel is zero, and $A/\operatorname{im} \overline{E}$ is not a singleton. The source generates a nonzero cokernel class c and an original representative $a \in A$ satisfying $[a] = c$ and $a \notin \operatorname{im} \overline{E}$. For every function $f : A \to \widehat{H}_s$, one has $\overline{E}(f(a)) \neq a$.

This total disposition is a noncomputable mathematical construction. The unsound branch extracts a relation witness from the original proof of unsoundness. The nonzero-kernel branch selects a nonzero member of the original kernel submodule. When the kernel is zero but faithfulness fails, a singleton cokernel would contradict the original failure of coverage. The cokernel is therefore not a singleton: subtract two distinct classes to obtain a nonzero class, then use surjectivity of the quotient map to obtain its representative. If that representative lay in the original range, its quotient class would be zero, contradicting the selected nonzero class. The kernel branch takes priority internally; a nonfaithful face need not supply both a kernel witness and a cokernel witness.

Complete proof for the direct consumer. Fix the original s and eliminate the four-branch disposition just generated by the source. In the faithful branch, $\phi\phi^{-1} = \mathrm{id}$ gives the recovery equation for every a. In the unsound branch, read $u \neq 0$ directly. In the kernel branch, read $k \neq 0$ and $k \in \ker \overline{E}$; kernel membership gives $\overline{E}(k) = 0$. In the coverage branch, take any f. If $\overline{E}(f(a)) = a$, then $f(a)$ together with this equality witnesses $a \in \operatorname{im} \overline{E}$, contradicting the exclusion of the original representative from the range. The last branch never assumes a linear recovery map, so the same exact representative also excludes every nonlinear recovery function.

The original material and residual projections are installed in the same source before the emitter. Installation preserves the original ledgerRoot, and the original emitter remains D.emit. Consequently, the disposition of the old row supplied together with any branch conclusion is still

$$\mathrm{entry}(s) \xrightarrow{\text{original receipt } \epsilon} \mathrm{entry}(D.\mathrm{update}(\epsilon)), \qquad \mathrm{next}(s) = D.\mathrm{update}(\epsilon).$$

Its same-debt budget readout is the nonincrease equality of the original transfer; the mathematical classification of residuals adds no strict payment. The four branch types and their elimination retain the original inputs above.

B All Old Types, General Splitting and Merging, and the Complete Remainder

B.1 Lifting Every Field of the Old Types

First fix the original complete native programme and its complete row list $I = \mathrm{Fin}(m)$. The extended index is $\mathrm{Fin}(m+1)$: castSucc retains every old position, and the final position is the new mathematical row. The old source and target entries, both selectors, and the complete receipts are reused at each position. Repeated content does not delete a position. Injectivity at both original ends follows only from the old IdentityScope certificate and the disjoint old/mathematical tags; coverage alone does not imply it.

General splitting and merging no longer require a target IdentityScope. Let V be the responsibility vocabulary of the mother source. In the extended vocabulary, content, residual, bearer, beneficiary, scope, event, admitted bearer/scope, and lineage all carry old/mathematical tags. Every old fibre in Type retains its original type: admission, authority, assumption, origin, discharge jurisdiction, and every progress, transfer, and terminal field. The embedding $j : \mathrm{Obligation}_V \to \mathrm{Obligation}_{V^+}$ adds the old tag field by field. The partial readback p eliminates the tags, and its old branch reconstructs the complete original record. Thus

$$p(j(a)) = \mathrm{some}(a),$$

including every original field in Type. The injectivity of j follows by applying p to an equality and eliminating some.

The children list of a mother split receipt becomes $\mathrm{map}(j)$ of the original children; the parents of a merge are treated identically. The preceding readback recovers the original member from each list member, so source ownership, anchor transport, lineage, and local discharge are exactly the original fields transported along tag equalities. Complete coverage is read from the original source fibre. When an old target has the same origin, injectivity of the old embedding recovers the original origin equality; apply the original covers_target and map the result into the new list. A mathematical target cannot have the tag of an old origin. For no phantom, recover the mother child from list membership, apply the mother's no-phantom condition to obtain the exact original target, and embed it. Strict budget decrease for each child is transported through the original budget-readout square. The merge proof is entirely symmetric and retains the discharge fields of every original parent.

Finally, analyse the tags of each target/source pair in the extended ledger. The old/old case uses the mother certificate's identity/split or identity/merge branch. The old/mathematical and mathematical/old cases are excluded by tags. The mathematical/mathematical case is the same independent row. This generates the complete restructuring certificate. Its original certificate comes from the same actual compiler image, transported through the original fold and target-occurrence equalities. The consumer reads that certificate; the target supplies no alternative classification or new coverage.

The complete continuing case takes as input the per-current Packet programme supplied by the original law. A Packet eliminates one of the native, relation, continued, or redirect branches of the

same generatedLedger and retains the original payload, complete evolution, original finite patch, and actual next state. Join the original mathematical action to that Packet. The mathematical paid branch uses its actual strict step; the mathematical settlement branch uses the original Idle evolution. The old destination and origin commute as

$$d^{+}(j_s e) = j_t(de), \qquad o^{+}(j_t f) = j_s(of).$$

These are complete dependent pairs carrying evolution witnesses, not merely equal entry names. The successor of the original owner row is obtained from $d(\text{owner})$ and transported along the target-emitted equality.

The generated patch still lists only one mathematical row explicitly; the same source remainder carries all old entries. The mathematical destination selector selects row zero. The old destination selectors and every origin selector select the remainder. The remainder event compiles to this complete ledger join and emits exact row evidence for every pair. Computing both complete Sigma types by tags proves that fold equals the complete ledger join. The complete mother restructuring certificate is lifted through this fold. The actual tick directly eliminates the same compiler result, so its complete destination, original projection outcome, and actual next state are equal simultaneously. A single Packet cannot replace the per-current programme; that input has been stated explicitly.

C Paid Syntax and Its Own Next Request

C.1 Execution and the Whole Ledger

Fix any language whose values form a family of additive commutative groups, a variable environment ρ, and a source expression e. Define the remaining count at each node by

$$r(c) = 0, \quad r(x) = 1, \quad r(e+f) = r(e)+r(f)+1, \quad r(Le) = r(e)+1, \quad r(B(e,f)) = r(e)+r(f)+1.$$

Variable binding, addition of two constants, and linear and bilinear operations each pay one primitive step. Contextual steps merely lift that generated step into the left or right subexpression. Checking every constructor proves that each actual step $s : e \to e'$ satisfies $r(e) = r(e') + 1$ and carries $\mathrm{ev}_\rho\, e = \mathrm{ev}_\rho\, e'$. The executor runs from left to right, evaluating subexpressions before their parent node. Induction on expressions gives

$$T_e : e \to \mathrm{const}(\mathrm{ev}_\rho\, e), \qquad |T_e| = r(e).$$

This trace itself generates relation words: the image of a one-step relation is $\delta_e - \delta_{e'}$, and concatenation takes their sum. Induction on trace length cancels the intermediate terms. Hence, for every paid prefix $T : e \to e_k$,

$$\partial\eta_T = \delta_e - \delta_{e_k}, \qquad \mathrm{ev}_\rho(\partial\eta_T) = 0.$$

The cancellation occurs in the relation image; the actual steps, their order, and the relation evidence remain in T.

The construction with no old owner takes the source root, original occurrence, and original input reader as its inputs. Extend the original support to a pair consisting of the original support and a mathematical state. The live ledger splits into all old entries and one independent mathematical responsibility. Its state retains (e_k, T_k), not just an endpoint value. The original selected action generates local settlement at a constant; otherwise, it emits the executor's first step. On the old branch, the complete write reads the same source remainder. On the mathematical branch, it writes the state $(e_{k+1}, T_k \cdot s_k)$ generated by this step. The finite patch contains only one mathematical row; its remainder actually compiles to the complete old-ledger evolution. Eliminating the branches of the dependent pairs on both sides identifies the patch destination and origin with the complete write.

Let $B = r(e)$. At the original process's kth visit, its mathematical budget is $B-k$. The base case has an empty trace. A payment appends one step and reduces the budget by one; local settlement preserves the complete state. Therefore

$$b_k = B - k, \qquad |T_k| + b_k = B.$$

The exact destination of the actual mathematical row is the mathematical row at the next visit. The disjoint old/mathematical claim tags make this same-debt target unique. The row evolution generates lineage, and the strict decrease of the original action generates payment. There is an actual same-debt payment at every $k < B$. At $k = B$, the trace T_B has length B, its endpoint equals the original $\mathrm{ev}_\rho\, e$, and no payment macro remains for this mathematical budget. The local settlement obtained from execution continues to be consumed according to its own source contract.

The ten-node Fock instance also reads the actual current physical environment ρ_d. Its original trace belongs to the original environment ρ_0, the remaining budget is $10-(d+1)$, and trace length plus budget is ten. The complete physical pair is read from the current actual occurrence, rather than reconstructed from a paid value. If the current pending expression is e_d, the updated residual is exactly

$$\mathrm{physicalPair}_d - \mathrm{ev}_{\rho_d}(e_d).$$

At budget zero, e_d is already the constant of the original complete pair, so this value becomes the current-state pair minus the origin pair. The equality retains all components of both pairs and both actual environments.

The shared execution theorem and theorem with no old owner are cited by name as core Proposition 3.7 and Theorem 6.10 (S0.1–S0.3 and S1). The complete ledger and physical-current consumers are established above.

C.2 Integers, Substitution, Cochains, and Calls

Integer coefficients. Define $N_0(e) = 0$ and $N_{a+1}(e) = e + N_a(e)$. Induction on a gives evaluation $a\,\mathrm{ev}(e)$ and budget $a(r(e)+1)$. A negative integer $z = -(a+1)$ executes one additional original

linear negation node, so

$$c_z(e) = |z|(r(e) + 1) + \mathbf{1}_{z<0}.$$

The finite support list L of the source word is generated in its original order: the empty list is constant zero, and $(e, z) :: L$ gives $E_z(e) + F(L)$. A further induction on the list gives

$$r(F(L)) = \sum_{(e,z)\in L} (c_z(e) + 1), \qquad \mathrm{ev}(F(L)) = \sum_{(e,z)\in L} z\,\mathrm{ev}(e).$$

The executor produces a trace of exactly this budget. Equal or zero values after coefficient summation do not delete actual expressions or their charges.

Binding bodies. Let θ map original variables to target source expressions. Fix a target environment Θ and define the original environment by $\rho(x) = \mathrm{ev}_\Theta(\theta(x))$. Substitute the original steps and traces in this ρ; the target trace executes in Θ. The substituted trace of an original step is generated constructor by constructor. The bind x branch executes the entire $\theta(x)$; a constant-operation branch still executes its corresponding step; a contextual branch only lifts the generated trace into the original context. Original concatenated traces are combined by append. The substituted trace therefore actually connects $e[\theta]$ to $e'[\theta]$, and its relation boundary still telescopes to the difference of the two endpoints. At a constant endpoint,

$$|T[\theta]| = r(e[\theta]).$$

The normal value and whole-ledger/stage receipts generated by the mother engine from the same raw pair are read back through complete HEq. Executing the binding body does not charge for its entire execution as a single callback.

Ordered action words. Let $w = \ell_1 \cdots \ell_n$. The original sourceProgramme adds one linear node per letter, one node for the original variable, and one for the model projection, giving cost $n+2$. A query that retains the complete prefix and then applies a subsequent action adds one node, giving cost $n+3$. The joint programme for E and the complete future model has total budget $2n+5$. The source-effect history recursively retains the point before each letter action, the original one-step residual, and its corresponding actual execution trace. The empty word costs five execution steps and each single letter costs seven. Thus historyCost satisfies

$$h([]) = 5, \qquad h(\ell w) = 7 + h(w), \qquad h(w) = 7n + 5.$$

The complete inventory embeds both the whole-word trace and the history for each letter; $2n + 5$ and $7n + 5$ are charge identities for two original programmes. Propagating the residual through the suffix gives $R(\ell w, x) = U_w r(\ell, x) + R(w, A_\ell x)$; induction yields the complete sum of ordered residuals. Cancellation of endpoint values removes neither trace nor any original position.

Every cochain coordinate. Given an expression e, take the List $[\mathrm{updated}(e), \mathrm{old}(e)] +$ $+\mathrm{mixedTerms}(e)$ and index it by Fin of its length. The position map for mixed terms, $i \mapsto i + 2$, is injective; repeated expressions retain distinct positions. The vector query adds a coordinate embedding and an accumulation node for each term. The three families read back the updated value, the old value, and every mixed value, respectively. The original mixed-term update theorem

consequently becomes an exact equation for this same vector. Its charge is the sum of all original term charges plus twice the List length. The original execution generates its own trace and relationWords, which are written into typed Born.

Prefixes and batches. Take the original AST source seed e, with $e_i = e[\theta]^i$. Pair lifting preserves the budget of the ith original prefix, so the actual T_i has length $r(\tilde{e}_i)$. Put

$$\Sigma_n = \sum_{i=0}^{n} r(\tilde{e}_i).$$

The source batch uses $\mathrm{Fin}(n+1)$ and adds a coordinate embedding and an accumulation node at every position. Induction on its list gives exactly

$$|T_{\mathrm{batch}}| = \Sigma_n + 2(n+1).$$

Every coordinate reads back the complete old/effect pair of its original prefix. All prefix exposures enter the complete inventory through source seeds.

Actual calls. Let B be the finite variable bound of this original source seed. Calls are indexed by $\mathrm{Fin}(B+2)$. The jth call, at tick $c+j$ of the original Future runtime, reads its own complete callFrame, original input, query, Result, and execution trace. The observations are the original environments of these inputs. snapshotOld reads ρ_{c+j}, snapshotIncrement reads $\rho_{c+j+1} - \rho_{c+j}$, and positions $j > B$ take zero according to the original finite-variable contract. Eliminating the bounded branch at each variable position identifies the actual node environment readback with the original finiteOld/finiteIncrement, so the batch query values are equal. Each call also has its own charge:

$$\mathrm{callCharge} = \sum_{j=0}^{B+1} |T_j| = \sum_{j=0}^{B+1} r(\mathrm{Raw}_j) = \mathrm{callCost}.$$

This identity differs from $\Sigma_n + 2(n+1)$: the latter charges the snapshot batch's own AST, whereas the former charges every call actually issued. For each call, OwnerFree.activePayment from its callReader generates the strict debit and original lineage of every primitive step. The new batch trace has its own same-debt payment and written Born/cofinal embedding.

Actual children. Execute only the children selected by the original old and next branches. A child with no generated successor is constant zero and has charge zero. For an actual child, the original input of its subtree costs $4m$, and output injection adds one, so childCost is $4m+1$. The main AST sums the original query, the old child, and the next child. Query-output injection costs one node, and the two additions cost two nodes. Hence

$$c_{\mathrm{main}} = c_{\mathrm{query}} + c_{\mathrm{oldChild}} + c_{\mathrm{nextChild}} + 3.$$

The old actor inventory, original child-relation trace, lifted childTrace, and current main paidTrace each enter stock through maps that preserve order and through source seeds. The next state that depends on the source continues through its original callbacks.

The default query follows the same discipline when it consumes actual subtrees. The joint fee of the complete Branch kernel and the relation AST equals the sum of the two fees plus 4;

extending the complete actor face adds one node; each selected child has its own original cost $4m_i$, becoming $c_i = 4m_i + 1$ after output injection. A branch without a source successor is constant zero, with cost 0. The main query's fee is the original query fee plus the two child fees plus 3. Expanding these syntax constructors node by node gives the equalities, and the actual execution traces give the corresponding lengths. Original actor inventory, child relation trees, and the main paid exposure enter the same inventory in original order. Complete Past, the new actor, and the new mathematical payment all enter the next inquiry; fees do not become free annotations at the default call.

C.3 Admission of the Original Mathematical Row at Every Count

The mathematical compiler's initial patch selects its own mathematical row. Success of canonicalGeneratedEntryRow follows from the selection equality of this actual patch; initialRow then generates the sealed initial authority. Recurse on k: given (e_k, a_k), the next entry is canonicalTargetEntry of the same complete ledger write, and the next authority is a_k.next. There is no readmission from the target side. This yields the original row and authority at every count.

The complete material retains $M = (\rho, e, e', T, \text{owner}, \varepsilon)$. The next residual request is $e + (-e')$, executed in $\rho + \varepsilon$. Soundness gives old value zero, and the update equation gives new value $\text{eff}(e) - \text{eff}(e')$. Its AST budget

$$r(e) + r(e') + 2 > 0$$

ensures that the first action is an original primitive payment step. That step, the original registered input, and the source authority generate the responsibility birth programme. Its compileInquiry equals debtAdmission at the same exact event. The target's firstDestination, every old projection outcome, and literal next are fields of this same generatedAction. For any original Frame, programme owned by the source, and independent LowVar, Shared continues the actual query/answer/next at every offset, while reading nonrefill and payment well-foundedness without a total law along the mathematical row. When the increment of AC's fixed environment is zero, the request still retains the original raw body and pays its syntactic responsibility of $B + 2$.

D Typed Inventories, Content Updates, and Complete Cofinality

D.1 Dependent Readback of the Complete Inventory

The complete material contains physicalRaw/lowRaw, both paid states and execution traces, binding for each variable, the migrated trace, and written. At every supplied occurrence, the original stockLaw directly outputs this complete dependent record. The inherited outcome of the mother target is HEq to the source outcome. This same HEq gives equality of payload types. Cast the original target outcome along that type equality, then eliminate the sum type: inactive is PEmpty, and the active projection returns the original completeMaterial. Composing the heterogeneous equality of the cast with the original HEq yields ordinary equality. The stock read out is therefore exactly the original record. No type is inferred from a field name or an equal value.

Physical events migrate into the low language through the original binding, expression by expression and relation word by relation word. The old and low branches inject into the left and right source seeds, retaining multiplicity. The source's original written exposure and actual query exposure are first identified by complete tree equalities, then embedded into Born. For the positions in the current query trace, the three original source-seed joins give

$$\iota(j) = j + |H_{\mathrm{old}}| + |H_{\mathrm{physical}}| + 3, \qquad H_{\mathrm{Born}}[\iota(j)] = H_{\mathrm{query}}[j].$$

Cancelling the common offset in the natural numbers proves injectivity, and adding that offset proves strict order preservation. Equal values cannot delete distinct positions. This exact offset belongs to the named query-to-Born tree shape.

The complete epoch inverse writeback has a separate source fold. Stage d first executes the current syntax in the literal next environment, generating nextTrace/nextWord, then performs its paired inverse read. written(d) retains the old pair and this new paid exposure. historyWritten(0)=written(0), and historyWritten(d+1) is the source seed joining historyWritten(d) and written(d+1). Induction on the last stage shows that every original event of every $0 \le d \le$ depth belongs to historyWritten(depth). Eliminating the actual next branch shows that mathNext preserves the original inventory and nextBorn takes the same sourceWritten. The complete inventory therefore includes every epoch, rather than retaining only the latest trace or normal value.

D.2 The Actual Content of Birth

The actual result of pair lifting is $(\mathrm{ev}_{\rho}\, e, \mathrm{eff}_{\rho,\varepsilon}\, e)$. The source update identity makes the sum of these components exactly $\mathrm{ev}_{\rho+\varepsilon}\, e$. The original decoder of the actual birth-environment readout reads this paid pair, and the environment field of nextBorn is its readout by definition. Birth therefore advances the actual content while also carrying the old scalar and pair inventories and the complete paid exposure of this occurrence.

The original History programme's output in the active environment becomes the next environment. Applying the same construction to this literal Born gives a second actual programme output. The original history(stage) seed already supplies the active environment at stage+1, so the next birth reads history(stage+2). Each update comes from its own original trace and occurrence; renaming one result does not create a second execution.

The typed child of the source family is generated by the original Move recursion through nextSeed/nextCfg. Admission.generatedAction supplies the target root, complete first ledger write, old projections, source validity, and next reader through the same interface. Born.next_environment_source completely identifies the environment with physicalNext of the same occurrence. Environments at different occurrences are connected along their own equalities. The Some branch with an actual decoder and the none compatibility branch are each used according to the original code.

D.3 The Exact Distance of the Original Type Receipt

Fix any valid source programme, original Frame, and starting index i. Let B_i be the remaining count of the actual pending AST. The receipt returns $(d, s,$ actual action at $i + d =$ settled$(s))$

in Type, together with $d \le B_i$. Eliminate the original action: mathematical settlement returns distance zero; the paid branch uses exact mathNext, reduces the budget by one, recurses only on this next Frame, and adds one to the distance. The paid-branch equality and index equality $(i+1)+d' = i+(d'+1)$ transport the endpoint together.

Induction on $h \le B_i$ gives $B_{i+h} = B_i - h$. If $h < B_i$, the budget is positive, so the mathematical-settlement branch is impossible and one obtains the original actual paid Step. The receipt's endpoint budget is zero, and $d \le B_i$; substitution yields $d = B_i$. Appending the trace step by step gives every paid prefix, and soundness gives the exact endpoint value. At the same mathematical-settlement event identified by the receipt, the original compiler actually emits debtAdmission, and its next node is completely identified with nextBorn. D.1 establishes the position embedding into the complete stock.

D.4 The Same Runtime and Every Future

The original StockTarget compile function matches the actual action directly: the paid branch produces answered and takes mathNext; mathematical settlement produces debtAdmission of the original birthProgram and takes nextBorn. Expanding the compiler match in each branch gives its branch equality. successor_valid applies the original directlyAnswered or debtAdmission law, respectively, and identifies the next presentation using the complete targetRoot/targetCurrent equalities. Mathematical settlement cannot be erased into the answered branch.

Stage retains the language grade, complete packet, configuration, and language; continuation also retains the original paid Frame. Induction identifies the actual node of the unique runtime at tick t with the presentation of this Stage at t. Every query, branch-specific receipt, and complete ledger is read from the original tick. Language grade does not replace the tick.

Let $d(t)$ be the distance from tick t to the first admission at or after that tick. The initial node is an admission, hence $d(0) = t_0 = 0$. Eliminate Stage at any starting point. Admission gives distance zero. In continuation, D.3 supplies B paid prefixes and the mathematical-settlement endpoint; the next original tick then enters admission, giving distance $B+1$. Every intermediate Frame equals the Frame at the corresponding index of the same runtime. Put

$$t_0 = d(0), \qquad t_{j+1} = t_j + 1 + d(t_j+1), \quad \ell_j = 1 + d(t_j+1).$$

Then $t_j + \ell_j = t_{j+1}$ and $t_{j+1} > t_j$. Induction gives $t_j \ge j$, establishing cofinal coverage of every bound. historyAt(j) is the complete runFrom at the original tick t_j+h; it includes the admission tick and every paid or mathematical-settlement tick between stops. packetAt(j+1) equals the complete packet of the original receiver→receipt→birth chain. The language grade increases only at this actual birth.

At every stop, the material M_j retains the original input, environment, paid state, complete trace, and original owner row. The four fields of the installed original input are transported through the identity of the original complete current. The trace relation-word image $b_j = \delta_{e_j} - \delta_{e'_j}$ lies in the relationClosure of the same paid history. All six faces arise from that same material, and literal-word recovery holds for every complete word. Both the old σ and the σ generated by reexecution in the new environment retain their own place in the inventory.

At stop j, on the complete native word module $\mathcal{W}_j$, literal sourceWordAction first performs pair lifting, then transports along the language/complete-packet equalities of the actual next packet to obtain $S_j : \mathcal{W}_j \to \mathcal{W}_{j+1}$. Original inventory members embed through bornStock, so the word actually acted on enters the next original inventory. receiverRead first transports back through these equivalences, then reads the original receiver environment, giving

$$\mathrm{receiverRead}_j(S_j w) = (\mathrm{ev}_{\rho_j^{\mathrm{receiver}}}\, w, \mathrm{eff}_{\rho_j^{\mathrm{receiver}},\varepsilon_j^{\mathrm{receiver}}}\, w).$$

This is the original receiver-environment square; environments at different language grades need not be equal beforehand. The source_span of the complete paid state continues to advance on the original ticks as in the preceding part of D.4.

Finite support in the common carrier $\mathcal{W} = \bigoplus_j \mathcal{W}_j$ remains finite under $j \mapsto j+1$, and linear extension gives S. The local old/effect reader d_j has domain $\mathcal{W}_j$. The site projection $\mathrm{pr}_j : \mathcal{W} \to \mathcal{W}_j$ extends it to the direct-sum reader $D_j = d_j \circ \mathrm{pr}_j$. All permitted letters are selections from PUnit for the original single action. For any finite word a in these letters, define

$$K_{\mathrm{fut}} = \bigcap_a \bigcap_j \ker(D_j S_a).$$

Prepending one original letter stays within the quantified words, so this kernel is closed under S and the quotient generates modelAction. Equality of classes is equivalent to equality of every word/site observation. Any submodule that is currently invisible and invariant under S lies in this kernel by induction on word length; hence this kernel is maximal. The empty word includes the current readout, placing the future kernel inside the original native completion kernel and producing a valid nativeRestriction. Through it, modelAction reads the next value of the original S. The original complete paid state, CommonWords, receiver environment, and complete future model each retain their own type and are consumed together by the actual consumer.

D.5 The Difference Word's Own Charge and the Total of Each Receipt

The boundaries b_Ψ, b_μ of the two complete original traces at the next occurrence each belong to its actual history relation closure. Their readouts at the same scope are $r - c(\Psi, 0)$ and $r - c(\mu)$, where the original writer has $\mu = (\Psi, \sigma)$. Subtraction gives the actual scope readout

$$(r - c(\Psi, 0)) - (r - c(\mu)) = c(0, \sigma).$$

The source difference word independently generates the coefficient AST of C.2 and executes it at its own cost, producing its own Result and complete trace. The supplied material of the complete source coface enters the actual $k+2$ packet through mother admission; the original whole ledger and every old projection are retained simultaneously.

The effect and updated inverse of the same Producer.result read this new trace. Every new paid event first enters pairWritten, then enters actual jointStock through native_paid_in_stock and stock_in_joint. Composing these three named embeddings gives membership for each occurrence. The actual consume uses this carrier, the original outer reader, inner query, and complete ledger-next together. Value cancellation does not delete an occurrence.

Total-value writeback starts from the original complete material of the same occurrence. The original paid endpoint is const(p), with $p = \mathrm{ev}_{\rho}\, e$. The new native request $e - \mathrm{const}(p)$ completes in the actual afterEnv $\rho + \varepsilon$, with value

$$n = \mathrm{ev}_{\rho+\varepsilon}\, e - p, \qquad p + n = \mathrm{ev}_{\rho+\varepsilon}\, e.$$

The original cfg changes only nextEnvironmentReadAt owned by the source, preserving the reader and both inventory callbacks. The actual receipt selects the mathematical-settlement event. In that branch, the original decoder calls write(p+n), so the actual Born writes this receipt's total. If the native value is zero, it still writes p. The next packet rereads its own actual actionResult to obtain p^{+} and a fresh decoder, then repeats the equation above; it does not continue to use p. The shared proofs are cited by name as process-core Theorems 8.14/8.15 and S2.3/S5.2. The complete H0 source-environment repair is cited under the identity of its actual public implementation.

E The Source Programme Interface of the Original Long Calculation

E.1 The Original Arithmetic Linear Calculation

For a zero-observation input specified in Section 14.1 of the main text, the original nontrivial coordinate and right-half proof fibre provide the original q_0, operator R, and exact Xi order m. The source AST is $e_0 = x$, $e_{k+1} = R(e_k)$, and the environment contains only the original q_0. Induction gives evaluation $R^k q_0$ and syntactic charge $k + 1$. The executor therefore generates the actual targetState, value $R^m q_0$, and complete trace of length $m + 1$. The component is installed before the original emitter, and the readout at the same occurrence in the closed facade directly extracts this Result. Relation, inverse, and cochain each consume this same trace.

The original lower ledger, emitter, and physical next are retained through the installation square of the same projection; $m+1$ is the responsibility scale of the original AST. The actual physical tick separately reads the original root's receipt. The domain's original Xi consumer supplies physical membership after the actual value; its analytic derivation belongs to RS. WA34 already uses this actual value to generate the constEnv increment, the next R residual, and the complete R/split pair, so feedback is already supplied. This original programme interface supplies the actual material for the complete window and next correction of Section 14 in the main text.

References

[1] Jian Gao. *The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes.* Version 2, 2026. DOI: 10.5281/zenodo.23297175. Main text and complete proof supplement.